

Hệ thống nhận diện biến số ứng dụng cho phát hiện phương tiện và đi sai làn

Báo cáo đồ án môn học — Thị giác máy tính

Hoàng Thị Hoạt Đỗ Công Trí Nguyễn Đức Mạnh

Trường Đại học Xây dựng Hà Nội — Khoa Công nghệ thông tin
GVHD: GV. Nguyễn Đình Quý

Hà Nội, 2026

Bối cảnh & thách thức

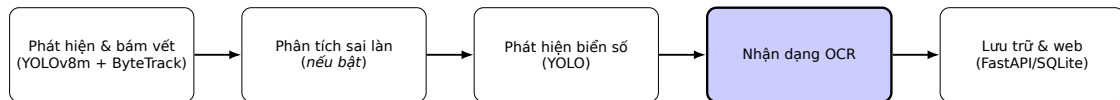
- Giám sát sai làn tại ngã tư chủ yếu thủ công: tốn người, dễ bỏ sót, thiếu khách quan.
- Ảnh camera khó: mờ động, rung, thiếu sáng, biển bị che/bám bụi.
- Biển số Việt Nam đa dạng: biển dài/vuông, 4 màu nền, ký tự Đ.
- Chỉ sai **một ký tự** là cả hồ sơ mất giá trị.

Mục tiêu

- Pipeline khép kín: video → phát hiện vi phạm → đọc biển → hồ sơ.
- Phát triển mô-đun **OCR tự huấn luyện** (trọng tâm); có demo chạy thực.

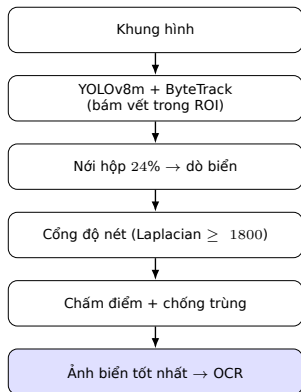
Ràng buộc

- Huấn luyện từ đầu (không pretrained).
- Chỉ dùng dữ liệu được cấp.
- GPU miễn phí Kaggle T4×2.



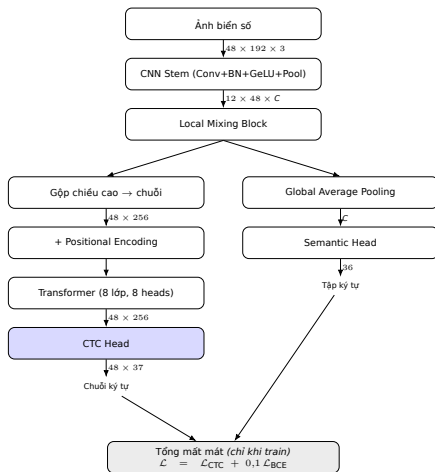
- ➊ Mỗi khung: YOLOv8m + ByteTrack phát hiện và bám vết xe (trong vùng quan tâm ROI).
- ➋ Phân tích sai làn theo vùng đa giác (bật/tắt tùy chế độ vận hành).
- ➌ Với xe cần xử lý: nối hộp 24%, dò biển bằng YOLO riêng, chọn khung nét nhất ($\text{Laplacian} \geq 1800$); lọc trùng ($\text{cô-sin} \geq 0,70$).
- ➍ OCR đọc biển rồi chuẩn hoá theo luật biển số Việt Nam.
- ➎ Lưu kết quả vào cơ sở dữ liệu và hiển thị trên web.

Hai khối nặng (biển số, OCR) bật/tắt theo **ba chế độ**: giám sát biển số / sai làn / phạt nguội tích hợp; khối **OCR** là trọng tâm.



Quy trình

- 1 YOLOv8m + ByteTrack bám vết xe, gán `track_id` (trong ROI; vết ≥ 4 khung).
- 2 Mỗi xe: nới hộp 24% rồi dò biển bằng YOLO riêng (conf 0,25).
- 3 Cắt biển, đo độ nét bằng phương sai Laplacian; loại nếu < 1800 .
- 4 Chấm điểm chất lượng, giữ khung biển **rõ nhất** cho mỗi xe.
- 5 Chống trùng: cô-sin $\geq 0,70$, cách ≤ 110 px, trong 45 khung.
- 6 Đưa ảnh biển tốt nhất sang khối OCR.



Lấy cảm hứng từ SVTRv2

- **Thân chung:** CNN Stem ($3 \times [\text{Conv } 3 \times 3 + \text{BN} + \text{GeLU}] + \text{Pool}$) + Local Mixing (Depthwise Sep. Conv + FFN).
- **Nhánh tuần tự** (chính): chuỗi 48 bước, $d = 256$, + Positional Encoding → Transformer 8 lớp · 8 heads (dropout 0,22) → CTC (37 lớp).
- **Nhánh ngữ nghĩa** (phụ, chỉ train): GAP → Semantic Head (36), giám sát phụ giúp hội tụ nhanh.
- Hai nhánh **không gộp** đầu ra; chỉ kết hợp ở hàm mất mát. Suy luận chỉ dùng CTC.
- Dùng Transformer thay BiLSTM: song song tốt + nắm phụ thuộc tầm xa.

Tập	Số ảnh
Train	9.752
Validation	1.219
Test	1.219
Tổng	12.190

Bộ ký tự: **36** (0-9, A-Z trừ W, và Đ); thêm blank của CTC $\rightarrow$ 37 lớp.

- **Sửa lỗi mã hoá nhãn** (Mojibake Đ/đ), chuẩn hoá in hoa, xuất CSV.
- **Sinh dữ liệu tổng hợp** $\times 5$ (≈ 58.512 mẫu), giả lập 5 màu biến (trắng/vàng/xanh/đỏ/xanh lá).
- **Tăng cường**: xoay $\pm 3^\circ - 15^\circ$, biến đổi phối cảnh, hạ phân giải, motion/Gaussian blur, nén JPEG (10-35).
- **Phân nhóm đa kích thước**: 40×160 , 48×192 , 56×224 .

Giai đoạn	Epoch	Dữ liệu chính	Mục tiêu
Nhập môn	1-40	Ảnh tổng hợp sạch, augment nhẹ	Học bảng ký tự, cấu trúc
Thử thách	41-160	Tổng hợp khó (blur, nghiêng) + ảnh thật	Chống nhiễu, mờ, lệch
Thực chiến	161-280	Tăng tỉ trọng ảnh thật	Tối ưu trên dữ liệu thực
Sửa lỗi khó	281-340	Lấy mẫu khó	Sửa cặp ký tự dễ nhầm

- **Hàm mất mát có trọng số:** phạt nặng ký tự nét mảnh 1,8,2,3,7,9 (1,8-2,0); giảm số 0 (0,7).
- **Lấy mẫu khó:** tăng $\approx 45\%$ xác suất xuất hiện các ảnh từng đoán sai.
- Thực tế dừng sớm quanh **epoch 78** (nằm ở giai đoạn 2).

Cấu hình (config_used.json)

Optimizer	AdamW
Learning rate	3×10^{-4}
Weight decay	0,08
Warmup	2.400 bước
Batch size	128 (DDP, T4×2)
Dropout	0,22
EMA decay	0,9995
Synthetic ratio	5

Hội tụ (đơn mô hình)

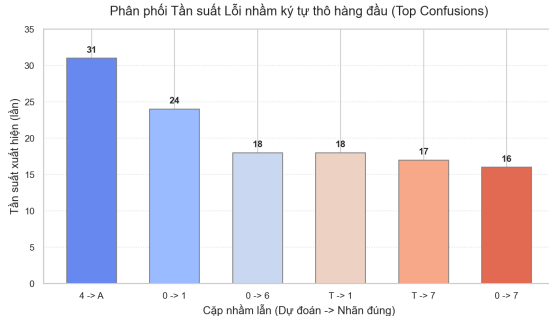
- Train loss: 3,45 → 0,05 (epoch 75).
- Train exact match: **95,00%**.
- Val thô (chưa hậu xử lý): **51,31%**.
- Tốc độ: 87–100 giây/epoch.

Lưu ý: chênh lệch train 95% vs val 51,31% cho thấy **quá khớp** đáng kể; 51,31% mới là năng lực thô thực sự.

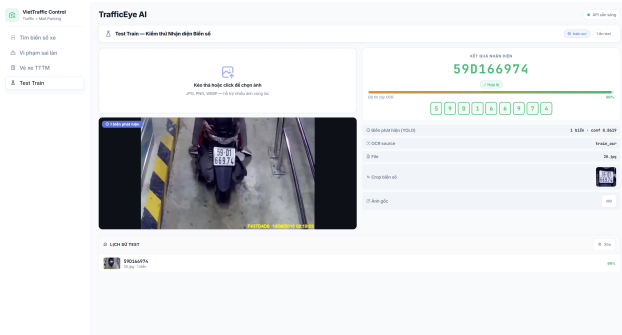
Chỉ số	Mặc định	Quét lại YOLO	Ý nghĩa
Character Accuracy	93,04%	93,06%	Tỷ lệ ký tự đúng
Full Plate Accuracy	74,82%	75,39%	Đúng 100% biển
Exact Match	—	912/1219	Số biển đúng hoàn toàn
Avg Edit Distance	—	0,5751	Số ký tự sửa trung bình
Latency (TB / P95)	74,56 / 136,74 ms	—	Thời gian/ảnh
Throughput	13,41 ảnh/s	—	≈ thời gian thực

- Tách rõ kết quả **thô** (51,31% trên val) và **toàn pipeline** (sau hậu xử lý).
- Cơ chế quét lại YOLO nâng full-plate từ 74,82% lên 75,39%.

- **Lỗi nhỏ chiếm đa số:** 142 ảnh sai 1 ký tự, 60 ảnh sai 2 ký tự $\Rightarrow$ ký tự đúng cao nhưng toàn biến thấp hơn.
- **Theo nhóm ảnh:** tốt nhất type1 87,4%, mb 84,6%; **khó nhất type2 chỉ 54,2%.**
- **Cặp nhầm phổ biến:** 4 $\rightarrow$ A (31), 0 $\rightarrow$ 1 (24), 0 $\rightarrow$ 6 (18), T $\rightarrow$ 1 (18), T $\rightarrow$ 7 (17).
- Nguyên nhân: nét tương đồng, mất viền tròn do mờ, loá sáng.



Kết quả mô hình & giao diện kiểm thử

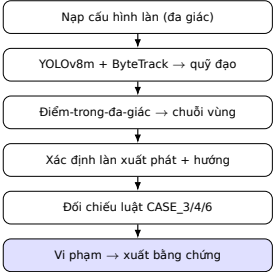


Trang **Test Train**: tải ảnh → YOLO cắt biển → OCR đọc theo từng ký tự kèm độ tin cậy.

Thật	Đọc	
30G58236	30G58236	Đúng
89LD00275	89LD00275	Đúng
21A08531	21A08531	Đúng
30E77379	30E77379	Đúng
30F07555	30F03055	Sai

Ca sai chỉ lệch ở cụm số, mã tỉnh và chữ vẫn đúng.

Mã	Làn xuất phát	Hướng thực tế	Kết luận
CASE_3	Làn rẽ trái	Đi thẳng	Đi thẳng trên làn rẽ trái
CASE_4	Làn đi thẳng	Rẽ trái	Rẽ trái trên làn đi thẳng
CASE_6	Làn rẽ trái	Rẽ trái	Lấn làn đi thẳng rồi rẽ trái



Quy trình

- 1 Nạp cấu hình làn (đa giác làn + vùng hướng).
- 2 Bám vết YOLOv8m + ByteTrack -> quỹ đạo từng xe.
- 3 Điểm-trong-đa-giác -> chuỗi vùng (vd 1 -> 2 -> 4 -> 3).
- 4 Suy ra làn xuất phát & hướng di chuyển thực tế.
- 5 Đối chiếu luật (Nghị định 168/2024) -> kết luận.
- 6 Xuất ảnh bằng chứng, ghi cơ sở dữ liệu.

Kết quả: 141 vết, 2 vi phạm — V001 xe máy (CASE_6), V002 ô tô (CASE_4).

Phương án	Seq/Plate	Char	CER	Điều kiện
Mô hình train (pipeline)	75,39%	93,06%	6,96%	Test, hậu xử lý
CRNN cơ sở	83,10%	95,68%	4,32%	Test, thô
CRNN v2 (điều chỉnh tham số)	93-95% (dự kiến)	—	~2,2%	chưa đo

Hướng phát triển = CRNN (CNN+BiLSTM+CTC), gọn, hợp dữ liệu nhỏ.

- **v2** chỉ là **điều chỉnh tham số** của CRNN cơ sở: lr, GroupNorm→BatchNorm, dropout, label smoothing, lấy mẫu cân bằng, augment mạnh, SWA.

- Ở mức **thô**, CRNN (83,10%) mạnh hơn train (51,31%).
- 75,39% của train là sau hậu xử lý, không so trực tiếp được.
- 93-95% của v2 chỉ là dự kiến, chưa đo.
- ⇒ Hướng CRNN đáng theo đuổi tiếp.

Điểm mạnh

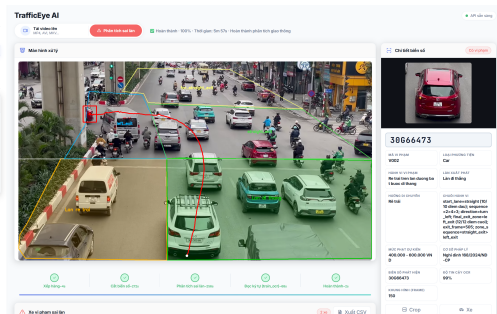
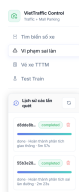
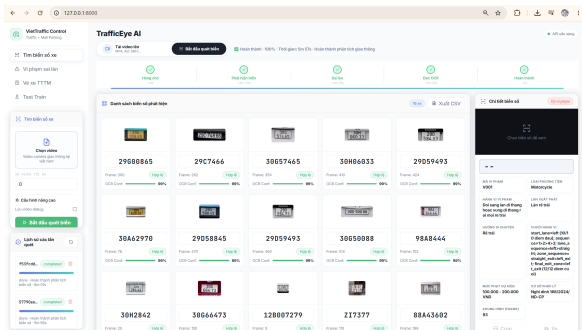
- Kiến trúc hiện đại, đa nhiệm.
- Train từ đầu, dễ tái lập.
- Tốc độ thời gian thực (≈ 13 ảnh/s).
- Tích hợp pipeline đặc thù VN.

Điểm yếu

- Quá khớp (train 95% vs val 51%).
- Phụ thuộc nhiều vào hậu xử lý.
- Yếu ở nhóm khó `type2` (54%).
- Chưa vượt CRNN ở mức thô.

Nguyên nhân gốc: độ phức tạp của Transformer lệch pha với quy mô dữ liệu nhỏ $\Rightarrow$ phải bù bằng dữ liệu tổng hợp và hậu xử lý.

Demo hệ thống — giao diện web



Trang *Tìm biển số*: danh sách biển đã phát hiện kèm độ tin cậy.

Trang *Vi phạm sai làn*: phân tích làn trực tiếp + chi tiết xe vi phạm (biển số, lỗi, mức phạt).

Kèm trang **Test Train** để tải ảnh và kiểm thử nhanh mô hình OCR.

Kết luận. Mô hình OCR tự huấn luyện chạy trọn vẹn trong pipeline giám sát, đạt **93,06%** ký tự / **75,39%** toàn biến ở thời gian thực; còn dư địa cải thiện ở năng lực nhận dạng thô.

Hướng phát triển

- Giảm quá khớp: tăng điều chuẩn, giảm tỉ lệ ảnh tổng hợp, kiểm soát phân phối.
- Nâng năng lực thô: label smoothing, lấy mẫu cân bằng; theo đuổi hướng CRNN.
- Bổ sung dữ liệu cho nhóm khó (type2).
- Tối ưu triển khai: ONNX/TensorRT cho thiết bị biên.
- Đánh giá mọi phương án trên cùng điều kiện đo.

Cảm ơn thầy đã lắng nghe!